Problem and implementation

Problem statement

IoT and smart building systems produce measurements, device alerts, and access records in a shared event file. The application converts that file into separate reports so each category can be analyzed using its own rules. It reads key=value fields, preserves spaces inside quoted values, and produces sensors.json, devices.json, and access.json. Invalid lines are skipped so one corrupt record does not stop the run.

Sensor reports group readings by name and calculate count, minimum, median, average, maximum, and population standard deviation. Device reports count events by severity and device, with severity order resolving frequency ties. Access reports calculate duration percentiles, count granted and denied requests, and identify users with at least three denials on the same door. Every category produces a report, including an empty JSON object when no records match.

Processing flow and responsibilities

Main uses ArgParser to read the CLI options and checks the input before processing begins. It also checks every report path against the input and the other reports. These checks account for symbolic links and hard links, and reject dangling report symlinks. Main then constructs AnalyzerPipeline with the parser, factory, aggregators, and writer. CLI and file errors produce stderr messages and a nonzero exit code.

AnalyzerPipeline reads one line at a time through BufferedReader. LineParser extracts the fields, and RecordFactory validates them before creating a typed Record. An unfinished quoted value invalidates the whole line. The pipeline uses an EnumMap to route each accepted record to its Aggregator, then passes the results and output names to OutputWriter.

Stats contains the shared calculations. SensorAggregator keeps the raw values, while JsonWriter uses Gson and BigDecimal to round outputs once and remove unnecessary decimal zeros. Stats uses scaled calculations when intermediate sums or squares overflow, and avoids overflow when averaging the middle values of an even-sized list.

Operational limits

The file is read one line at a time, but exact medians and percentiles require keeping values in memory. Memory use therefore grows with the accepted data. Reports are written in sequence, so a later write failure can leave earlier reports updated. Path checks do not prevent another process from changing files after validation. Numerical calculations retain the precision limits of doubles.

Pattern choices and trade-offs

Strategy: parsing, aggregation and output

I separated parsing, aggregation, and output through the LineParser, Aggregator, and OutputWriter interfaces. KeyValueParser, the three aggregators, and JsonWriter implement these strategies. Main selects the implementations and passes them to AnalyzerPipeline. Aggregator.handles() identifies the record type each strategy accepts, keeping type-specific calculations out of the processing loop.

Why it fits: the assignment requires support for future record types and formats. A single method would require less setup, but would combine parsing rules, classification, calculations, and serialization. Strategy gives each algorithm a defined responsibility and allows the parser or writer to be replaced through constructor wiring.

Benefits: an aggregation or format can change without rewriting the file-reading loop. Each implementation can be tested directly, while the pipeline and CLI tests check that the components work together. The interfaces stay small and use maps to pass fields and report data between stages.

Trade-offs: the interfaces introduce more classes and calls to follow. Map-based reports do not enforce their field names through the type system. Aggregators also rely on correct routing before casting a Record to its concrete type. Main must register each supported type, and aggregator state belongs to one analysis run.

Simple Factory: classification and validation

I kept record validation and creation in RecordFactory. Its classify method checks sensor, device, and access rules in that order, then creates SensorReading, DeviceEvent, or AccessRecord. Invalid or unmatched fields return Optional.empty(). Extra fields do not prevent an otherwise valid record from being classified.

Why it fits: creating records inside the processing loop would couple file iteration to required fields and value validation. A Simple Factory puts those decisions in one class. A registry of classifier objects would add setup for three short rules without changing the work those rules perform.

Benefits and trade-offs: the factory keeps validation consistent and testable without file input. Adding a record type still requires a classifier in the factory, an enum value, and wiring in Main. If a line matches more than one type, classification order determines the result. The implementation uses the Simple Factory idiom. It does not use subclass overrides to create records, as the GoF Factory Method pattern would.

Extending the implemented design

Network Logs: bytes_in, bytes_out, interface

Add NetworkLog implementing Record, using interfaceName because interface is a Java keyword. Add NETWORK to RecordType and a NetworkAggregator implementing Aggregator. Add validation and creation in RecordFactory, then register the aggregator under networks in Main. Existing parsing, aggregation, and output algorithms stay unchanged.

JSON Lines input

Add JsonLinesParser implementing LineParser. Use Gson to convert scalar fields into Map<String,String> and return an empty map for malformed lines. Main selects the parser; RecordFactory and the aggregators stay unchanged. Runtime format selection would also require changes to ArgParser and CliOptions.

CSV alongside JSON

Add CsvWriter implementing OutputWriter, defining headers, escaping, and empty reports. Add CompositeOutputWriter to call both writers, and select it in Main. Extend Main's collision checks to CSV paths before writing. The pipeline and aggregators stay unchanged. Add component and CLI tests for each extension.

UML class diagram

